VERIQTA

LEARN | PRACTICE | BUILD | GROW

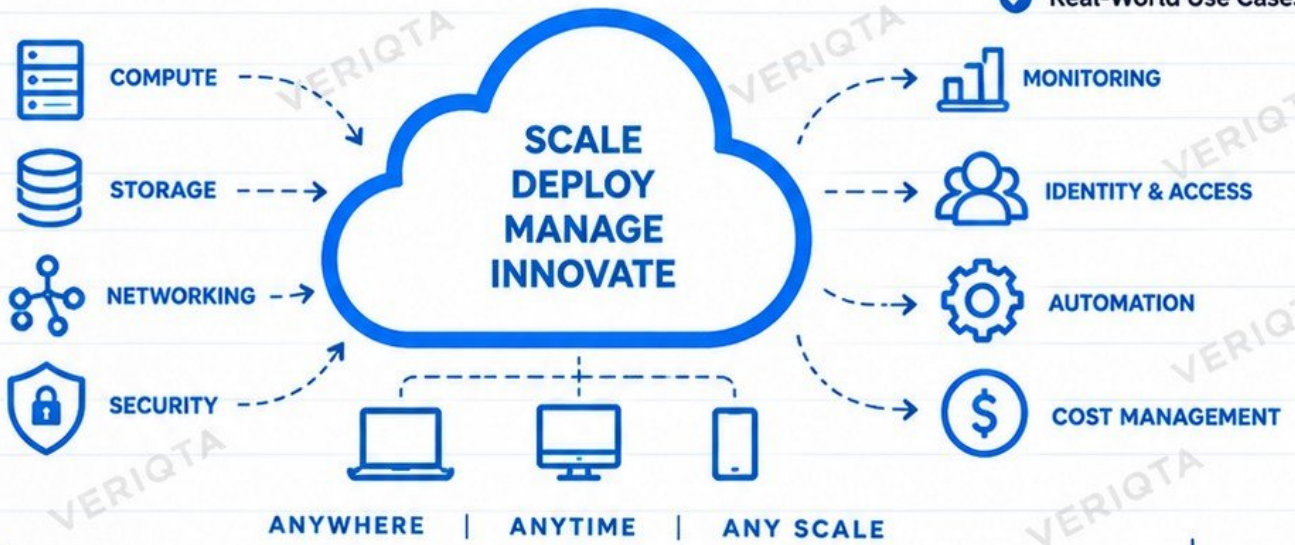
Better Engineers Brighter Futures

CLOUD COMPUTING HANDBOOK

From Cloud Fundamentals to Production Infrastructure

aws | Azure | Google Cloud

- Concepts
- Architecture
- Hands-On
- Best Practices
- Real-World Use Cases



Ideas Skills Progress VERIQTA

BUILD REAL SKILLS FOR A CLOUD-POWERED WORLD

Cloud Skills Real Opportunities

@veriqta | veriqta | veriqta | veriqta | @veriqta | @veriqta

CLOUD COMPUTING EXPLAINED,
WHAT ACTUALLY MAKES SOMETHING "CLOUD"

Same Knowledge Bigger Opportunities

1 WHAT IS CLOUD COMPUTING?

Cloud computing is the on-demand delivery of computing resources over the internet. You can provision, use, and manage servers, storage, networks, databases, and more without owning physical hardware.

2 TRADITIONAL vs CLOUD INFRASTRUCTURE

TRADITIONAL (ON-PREMISES)	CLOUD INFRASTRUCTURE
✗ Buy physical servers	✓ No physical hardware
✗ High upfront cost	✓ Pay as you go
✗ Manual setup and scaling	✓ Self-service provisioning
✗ Limited capacity	✓ Scale instantly
✗ You maintain hardware	✓ Provider maintains hardware
✗ Takes time to deploy	✓ Deploy in minutes

3 KEY CHARACTERISTICS OF CLOUD

ON-DEMAND RESOURCES
Get resources when you need them, not weeks later.

SELF-SERVICE PROVISIONING
Create and manage resources through a web console or API.

ELASTICITY
Scale up or down based on demand.

RESOURCE POOLING
Shared infrastructure used by many customers.

MEASURED USAGE
Pay only for what you use.

4 WHY COMPANIES MOVE TO THE CLOUD

- Reduce infrastructure costs
- Deploy faster
- Improve flexibility and scalability
- Increase reliability and disaster recovery
- Focus on core business instead of hardware management
- Access to advanced technologies (AI, analytics, etc.)

5 IMPORTANT CLARIFICATION

Cloud does not mean "someone else's computer" only. It is a complete set of infrastructure, platforms, and services delivered over the internet, managed by cloud providers in data centers worldwide.

6 MENTAL MODEL

APPLICATION

Your application (code, container, etc.)

→

CLOUD SERVICES

Compute, storage, databases, networking, security, and more

→

PHYSICAL INFRASTRUCTURE

Data centers, servers, networking hardware (owned by the cloud provider)

You use cloud services, but the underlying physical infrastructure is managed and maintained by the cloud provider.

7 JUNIOR ENGINEER TIP

Learn the infrastructure concept before memorizing provider service names.

Understand how compute, networking, storage, security, and other components work. This knowledge transfers across AWS, Azure, Google Cloud, and other providers.

Build Your Skills Build Your Future

Same Knowledge Bigger Opportunities

IAAS, PAAS, SAAS AND SERVERLESS

DIFFERENT LEVELS OF ABSTRACTION, SAME CLOUD

1 IAAS

Infrastructure as a Service

- Provides virtualized computing infrastructure
- You manage the OS, runtime, and application
- Example: AWS EC2, Azure VMs, Google Compute Engine

Best for: Maximum control and flexibility.

2 PAAS

Platform as a Service

- Provides a managed environment for your application
- You focus on code, not infrastructure
- Example: AWS Elastic Beanstalk, Azure App Service, Google App Engine

Best for: Faster development and less operational work.

3 SAAS

Software as a Service

- Complete software application delivered over the internet
- You just use it
- Example: Gmail, Microsoft 365, Salesforce, Dropbox

Best for: End users and business applications.

4 FAAS / SERVERLESS

Functions as a Service

- Run small pieces of code without managing servers
- Automatically scales
- You focus only on your code
- Example: AWS Lambda, Azure Functions, Google Cloud Functions

Best for: Event-driven applications and reducing infrastructure management.

5 WHO MANAGES WHAT?

✓ You manage ✗ Provider manages

COMPONENT	IAAS	PAAS	SAAS	FAAS / SERVERLESS
Application code	✓	✓	✗	✓
Runtime (e.g. Java, Python)	✓	✗	✗	✗
Operating system	✓	✗	✗	✗
Virtual machines / containers	✓	✗	✗	✗
Networking	✓	✗	✗	✗
Storage	✓	✗	✗	✗
Scaling	✓	✗	✗	✗
Patching and maintenance	✓	✗	✗	✗

6 CHOOSING THE RIGHT MODEL

- Need full control? → Use IaaS
- Want to focus on development? → Use PaaS
- Need a ready-to-use application? → Use SaaS
- Running event-driven code? → Use Serverless (FaaS)
- Consider cost, scalability, control, and team skills.

7 JUNIOR ENGINEER TIP

Learn the infrastructure concept before memorizing provider service names. Understand what you are using, who manages it, and why it fits your use case.

SAME CLOUD. DIFFERENT MODELS. MORE POSSIBILITIES.

Build Learn Share VERIQTA

Better Engineers Brighter Futures

PUBLIC, PRIVATE, HYBRID AND MULTI-CLOUD

DIFFERENT APPROACHES. SAME GOAL. REAL-WORLD USE.

1 PUBLIC CLOUD

- Infrastructure owned and operated by cloud providers.
- Access over the internet.
- Pay as you go.
- Highly scalable.
- Examples: AWS, Microsoft Azure, Google Cloud.

Best for: Speed, scalability and lower upfront cost.

2 PRIVATE CLOUD

- Dedicated infrastructure for one organization.
- Can be on-premises or hosted by a provider.
- More control and customization.
- Often used for sensitive or regulated workloads.

Best for: Security, control and compliance.

3 HYBRID CLOUD

- Combination of on-premises and public cloud.
- Keep some workloads on-premises, use cloud for others.
- Common in regulated industries.
- Requires good connectivity and management.

Best for: Flexibility and modernization at your own pace.

4 MULTI-CLOUD

- Use two or more public cloud providers.
- Avoid vendor lock-in.
- Choose best services from each provider.
- More complex to manage.
- Requires strong governance and cost control.

Best for: Flexibility, resilience and avoiding vendor lock-in.

5 ON-PREMISES INFRASTRUCTURE

- Infrastructure owned and managed by your organization.
- Located in your own data center or office.
- You are responsible for hardware, power, networking, and maintenance.
- Often used for legacy systems or highly regulated data.

Best for: Full control and specific compliance requirements.

6 WHY ORGANIZATIONS USE HYBRID ARCHITECTURES

- Keep sensitive data on-premises (e.g. financial, healthcare).
- Use the cloud for scalability and new applications.
- Modernize graduully instead of migrating everything at once.
- Optimize costs by running the right workload in the right place.
- Meet regulatory and data residency requirements.

7 WHY MULTI-CLOUD IS HARDER THAN IT SOUNDS

- Different tools, services and naming conventions.
- More complex networking and identity management.
- Harder to maintain consistent security and governance.
- Higher operational overhead and team skill requirements.
- Cost management can be challenging.
- Troubleshooting across multiple platforms takes more time.

8 CONNECTIVITY, DATA MOVEMENT AND COMPLEXITY

Connectivity Considerations
- Site-to-site VPN
- Direct Connect / ExpressRoute
- Inter cloud connectivity
- Latency and bandwidth

Data Movement
- Data transfer cost
- Latency
- Data synchronization
- Security and encryption

Operational Complexity
- Multiple consoles
- Different IAM models
- Monitoring and logging
- Consistent policies
- Skilled team required

9 REAL WORLD EXAMPLE: ON-PREMISES DATABASE + CLOUD APPLICATION

HOW IT WORKS
- Application runs in the cloud.
- Application connects to on-premises database through a secure network connection.
- Data stays on-premises for compliance.
- Users access the application over the internet.
- This is a common hybrid architecture.

KEY TAKEAWAY There is no single "best" model. The right choice depends on your business requirements, security, cost, compliance, and team capability.

Learn Build Grow VERIQTA

Same Knowledge Bigger Opportunities

REGIONS, AVAILABILITY ZONES AND GLOBAL INFRASTRUCTURE

UNDERSTAND THE BUILDING BLOCKS OF THE CLOUD

1 WHAT IS A REGION?

- A Region is a geographic area that contains multiple Availability Zones.
- Each region is a separate location (e.g. us-east-1, eu-west-1).
- Regions are isolated from each other.
- You choose a region based on latency, compliance and business requirements.

Example: AWS us-east-1 (N. Virginia), Azure East US, Google Cloud us-central1

2 WHAT IS AN AVAILABILITY ZONE?

- An Availability Zone (AZ) is a separate data center or group of data centers within a region.
- Each AZ has its own power, networking and cooling.
- AZs are physically separate from each other.
- Designed to reduce the risk of a single point of failure.

Example: AWS us-east-1a, us-east-1b, us-east-1c (all in the same region)

3 DATA CENTERS

- Physical facilities that house servers, storage, networking and cooling infrastructure.
- Each Availability Zone consists of one or more data centers.
- Built with redundant power, networking and security systems.
- You do not manage the data center. The cloud provider does.

4 EDGE LOCATIONS

- Edge locations are smaller facilities closer to users.
- Used for content delivery, DNS, caching and low-latency access.
- Part of global infrastructure (e.g. AWS CloudFront, Azure Front Door, Google Cloud CDN).
- Improve performance by serving content from a location near the user.

5 REGIONAL vs ZONAL RESOURCES

REGIONAL RESOURCES

- Exist across the entire region.
- Examples: VPC, managed databases, load balancers (some types), storage services (e.g. S3).
- Survive the failure of one AZ.

ZONAL RESOURCES

- Exist in a specific AZ.
- Examples: EC2 instances, virtual machines, disks, some databases.
- If the AZ fails, the resource is lost unless you have redundancy in another AZ.

6 LATENCY

- Latency is the time it takes for data to travel from a user to a service.
- Choose a region closer to your users to reduce latency.
- Use edge locations and CDNs for global users.

7 DATA RESIDENCY

- Data residency means your data is stored in a specific country or region.
- Important for compliance (e.g. GDPR, HIPAA, local laws).
- Choose regions that meet your legal and regulatory requirements.

8 HIGH AVAILABILITY

- Use multiple Availability Zones within a region.
- Distribute resources across AZs.
- If one AZ fails, your application continues to run.
- Use load balancers, auto scaling and managed services.

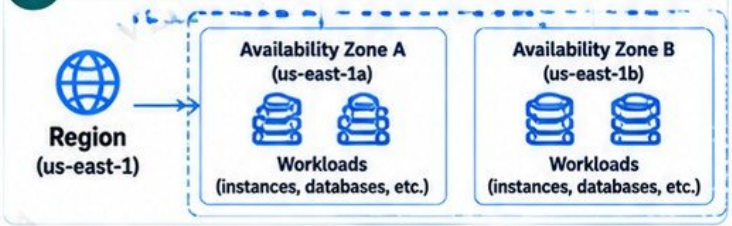
9 DISASTER RECOVERY

- Prepare for region-level failures (e.g. natural disasters).
- Use multiple regions.
- Replicate data across regions.
- Have a backup and recovery plan with clear RTO and RPO.

10 WHY TWO SERVERS IN ONE ZONE ARE NOT HIGHLY AVAILABLE

- Both servers share the same power, networking and physical infrastructure.
- If that AZ fails, both servers go down.
- High availability requires resources across multiple AZs.
- Think beyond individual servers.

11 ARCHITECTURE: REGION → AZ A + AZ B → WORKLOADS



KEY TAKEAWAYS

- A region contains multiple Availability Zones.
- Each AZ is a separate data center.
- Edge locations improve performance for global users.
- Use multiple AZs for high availability.
- Use multiple regions for disaster recovery.
- Choose regions based on latency, compliance and business needs.
- Two servers in one zone are not highly available.

Better Engineers Brighter Futures

CLOUD COMPUTE
VIRTUAL MACHINES FROM ZERO

UNDERSTAND. DEPLOY. CONFIGURE. RUN. IN THE CLOUD.

1 PHYSICAL SERVERS vs VIRTUAL MACHINES

Physical Server
- Dedicated hardware
- Runs one operating system
- Higher cost
- Less flexible
- Manual scaling

Virtual Machine
- Runs on shared hardware
- Multiple VMs on one server
- Lower cost
- More flexible
- Easy to scale

2 HYPERVISORS

A hypervisor is software that creates and manages virtual machines on physical hardware.

- **Type 1 (bare metal):** Runs directly on hardware (e.g. VMware ESXi, Hyper-V)
- **Type 2 (hosted):** Runs on an operating system (e.g. VirtualBox, VMware Workstation)

3 VM INSTANCES

- A VM instance is a virtual computer in the cloud.
- You choose the CPU, memory, storage, and operating system.
- Each instance runs its own OS and applications.
- You can start, stop, resize or delete it anytime.

4 vCPU

- A vCPU (virtual CPU) is a portion of a physical CPU.
- More vCPUs = more processing power.
- Used to run applications, handle requests and processes.

5 MEMORY (RAM)

- Memory is the short-term storage used by the OS and applications.
- More memory allows more applications and larger workloads.
- Choose based on your application requirements.

6 INSTANCE TYPES

- Predefined combinations of vCPU, memory, storage and networking.
- Optimized for different workloads (general purpose, compute optimized, memory optimized, etc.).
- Examples: AWS t3, m6i, c6i Azure D-series, Google e2

7 MACHINE IMAGES

- A machine image is a template used to create a VM.
- Contains the operating system and pre-installed software.
- Examples: Amazon Machine Image (AMI), Azure Image, Google Compute Image.

8 BOOT DISKS

- Boot disk (root volume) contains the operating system.
- Stored on block storage (e.g. EBS, Azure Managed Disk, Persistent Disk).
- You can choose the size, type (SSD/HDD), and enable encryption.

9 SSH ACCESS

- Secure Shell (SSH) is used to securely connect to your VM from your local machine.
- You need a key pair (private key and public key).
- Example: `ssh -i key.pem ubuntu@<public-ip>`
- Keep your keys secure and never share your private key.

10 PUBLIC AND PRIVATE IP ADDRESSES

Public IP
- Accessible over the internet.
- Used for SSH, web servers, etc.
- Can be static or dynamic.

Private IP
- Used only within a private network (e.g. VPC/VNet).
- Not accessible from the internet.
- Used for internal communication between resources.

11 USER DATA AND STARTUP SCRIPTS

- User data (startup scripts) run when the VM is first launched.
- Used to automate configuration (e.g. install packages, start services).
- Helps you deploy consistent environments.
- Examples: cloud-init (Linux), Custom Data (Windows).

12 MAJOR CLOUD PROVIDERS

Amazon EC2
- Wide range of instance types
- On-demand, reserved, spot instances
- Integration with AWS services

Azure Virtual Machines
- Flexible instance sizes
- Windows and Linux support
- Integration with Azure services

Google Compute Engine
- High performance VMs
- Sustained use discounts
- Integration with Google Cloud services

13 PRODUCTION AWARENESS

STOP TREATING SERVERS AS PETS

- Do not manually configure everything.
- Use automation and Infrastructure as Code.
- Design for replacement, not long-term attachment.
- Use immutable infrastructure where possible.
- Plan for scaling, monitoring, backups and recovery.
- Treat your infrastructure as cattle, not pets.

SAME INFRASTRUCTURE. BIGGER POSSIBILITIES.

CLOUD NETWORKING
BUILD THE NETWORK FIRST

Better Engineers Brighter Futures

NETWORKING IS THE FOUNDATION OF EVERYTHING IN THE CLOUD.

1 VIRTUAL NETWORKS

- A virtual network is an isolated network in the cloud.
- It provides a private network environment for your resources (servers, databases, etc.).
- You control IP addressing, subnets, routing and security.
- It works like your own data center network, but in the cloud.

2 CLOUD PROVIDER NETWORKS

AWS VPC	• Isolated virtual network • Custom IP range • Integrates with AWS services
Azure VNet	• Isolated virtual network • Custom IP range • Integrates with Azure services
Google Cloud VPC	• Isolated virtual network • Custom IP range • Integrates with Google Cloud services

3 CIDR BLOCKS

10.0.0.0/16

- CIDR (Classless Inter-Domain Routing) defines the IP address range for your network.
- Example: 10.0.0.0/16 (65,536 IP addressee)
- You divide this range into smaller subnets.
- Plan your IP addressing before creating resources.

4 PRIVATE vs PUBLIC IP ADDRESSES

PRIVATE IP

- Used within the virtual network.
- Not accessible from the internet.
- Examples: 10.0.1.10, 192.168.1.10

PUBLIC IP

- Accessible from the internet.
- Used for resources like load balancers or bastion hosts.
- Examples: 18.212.10.5, 52.14.33.8

5 SUBNETS

- A subnet is a smaller range of IP addresses within your VPC/VNet.
- You can create multiple subnets in different Availability Zones.
- Subnets can be public or private depending on their route table.
- Example: 10.0.1.0/24

6 PUBLIC vs PRIVATE SUBNETS

PUBLIC SUBNET

- Has a route to the internet (Internet Gateway).
- Used for resources that need internet access (e.g. web servers, load balancers).

PRIVATE SUBNET

- No direct internet route.
- Used for internal resources (e.g. application servers, databases).
- Uses a NAT gateway for outbound internet access (if needed).

7 ROUTE TABLES

- A route table controls where network traffic is sent.
- It contains routes (rules) for different destinations.
- Each subnet is associated with a route table.
- Example routes:

10.0.0.0/16	→ local
0.0.0.0/0	→ Internet Gateway

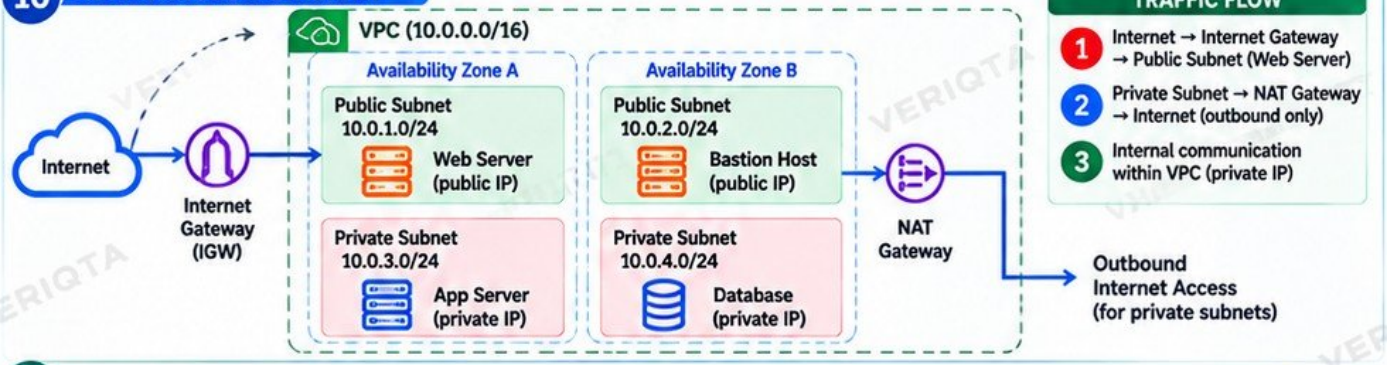
8 INTERNET GATEWAYS

- An Internet Gateway (IGW) allows your VPC/VNet to communicate with the internet.
- It enables resources in public subnets to have internet access.
- One IGW per VPC.

9 NAT (NETWORK ADDRESS TRANSLATION)

- A NAT gateway allows resources in private subnets to access the internet (outbound only).
- It keeps your private resources secure (no direct inbound access from the internet).
- Managed by the cloud provider (e.g. AWS NAT Gateway).

10 NETWORK FLOW DIAGRAM



TRAFFIC FLOW

1. Internet → Internet Gateway → Public Subnet (Web Server)
2. Private Subnet → NAT Gateway → Internet (outbound only)
3. Internal communication within VPC (private IP)

11 JUNIOR ENGINEER TIP

Most "cloud problems" eventually require networking knowledge.
Learn how networks, subnets, route tables and gateways work. It will help you troubleshoot faster and design better, more secure architectures.

Learn Practice Build Grow VERIQTA

Better Engineers Brighter Futures

FIREWALLS, SECURITY GROUPS AND NETWORK ACCESS

CONTROL TRAFFIC. PROTECT RESOURCES. KEEP IT SECURE.

1 PORTS AND PROTOCOLS

- A port is a number (0–65535) used to identify a service on a server.
- Common protocols: TCP, UDP, ICMP.
- Examples:

Service	Port	Protocol
HTTP	80	TCP
HTTPS	443	TCP
SSH	22	TCP
RDP	3389	TCP
DNS	53	TCP/UDP
MySQL	3306	TCP
PostgreSQL	5432	TCP

2 INBOUND vs OUTBOUND TRAFFIC

Inbound Traffic

- Traffic coming into your resource from outside.
- Must be explicitly allowed.
- Example: Users accessing your web server (port 80/443).

Outbound Traffic

- Traffic going out from your resource to the internet.
- Often allowed by default.
- Example: Your server updating packages, calling an external API.

3 STATEFUL vs STATELESS FILTERING

Stateful Filtering

- Remembers the state of connections.
- Automatically allows return traffic.
- Used by Security Groups (AWS) and some firewalls.
- Easier to manage.

Stateless Filtering

- Does not remember connection state.
- You must explicitly allow both inbound and outbound traffic.
- Used by Network ACLs (AWS) and some firewalls.
- More control but requires careful configuration.

4 SECURITY GROUPS

- Acts as a virtual firewall for your cloud resources (e.g. EC2).
- Stateful (allows return traffic).
- Rules define allowed inbound and outbound traffic.
- Applied to specific resources (e.g. instances, load balancers).
- Example: Allow TCP port 22 from your IP for SSH.

5 NETWORK SECURITY GROUPS

- Used in Azure (NSG).
- Controls inbound and outbound traffic.
- Can be stateful or stateless depending on the configuration.
- Applied to subnets or network interfaces.
- Example: Allow HTTP (80) to your web server subnet.

6 FIREWALL RULES

- Define what traffic is allowed or denied.
- Based on source, destination, port, and protocol.
- Can be network firewalls, host firewalls, or cloud firewalls.
- Use the principle of least privilege.
- Regularly review and remove unnecessary rules.

7 NETWORK ACL CONCEPTS

- Network ACLs (NACLs) act at the subnet level (AWS).
- Stateless (must allow both inbound and outbound).
- Rules are evaluated in order by rule number.
- Can allow or deny traffic.
- Useful for an additional layer of security.
- Example: Deny all traffic except specific ports.

8 SOURCE AND DESTINATION

- Source: Where the traffic comes from (e.g. IP address, CIDR range).
- Destination: Where the traffic is going (e.g. IP address, CIDR range).
- Rules can be based on source, destination, port, and protocol.
- Example: Allow traffic from 203.0.113.10 to port 443 on your server.

9 WHY 0.0.0.0/0 MATTERS

- 0.0.0.0/0 means "any IP address" (anywhere on the internet).
- Allows access from all public IPs.
- Useful for public services (e.g. web servers), but risky if not required.
- Better practice: Restrict to specific IP ranges when possible.
- Example: Allow 0.0.0.0/0 on port 80 for a public website, but not on port 22 (SSH).

10 CONNECTION REFUSED vs TIMEOUT

Connection Refused

- The server is reachable, but the port is closed.
- Usually means no service is listening or a firewall is actively rejecting it.
- You get a fast response.

Connection Timeout

- No response from the destination.
- Usually caused by a firewall dropping the traffic or a network routing issue.
- You wait and eventually get a timeout error.

11 TROUBLESHOOTING INACCESSIBLE CLOUD APPLICATIONS

1. Check if the service is running on the correct port.
2. Verify security group / firewall rules.
3. Check network ACLs (if applicable).
4. Confirm the source IP is allowed.
5. Test connectivity using tools like ping, telnet, nc, or curl.
6. Review logs (e.g. VPC Flow Logs, firewall logs, application logs).
7. Check routing, subnets, and internet/NAT gateways.

12 SECURITY REMINDER

EXPOSE ONLY WHAT IS REQUIRED.

- Follow the principle of least privilege.
- Regularly review and remove unnecessary access.
- Restrict access to trusted IP ranges.
- Use private subnets for internal resources.
- Monitor and log network traffic.
- Security is everyone's responsibility.

SECURE NETWORKS. RELIABLE APPLICATIONS. STRONGER ENGINEERS.

Learn Practice Build Grow VERIQTA

DNS, LOAD BALANCERS AND APPLICATION TRAFFIC

Good Infrastructure Runs Applications

Connect users to your applications, reliably and securely.

1 WHAT DNS DOES

- DNS (Domain Name System) translates human-friendly domain names into IP addresses.
- It helps users find the right server without needing to remember IP addresses.
- It works like a phonebook for the internet.

Example:
www.example.com → 93.184.216.34

2 DOMAIN → IP RESOLUTION

1. You enter a domain name in your browser.
2. DNS looks up the IP address for that domain.
3. Your browser connects to that IP address.

3 PUBLIC vs PRIVATE DNS

PUBLIC DNS

- Resolves domain names on the internet.
- Used for public websites and internet-facing services.
- Example: www.google.com

PRIVATE DNS

- Resolves domain names inside a private network (VPC/VNet).
- Used for internal applications and services.
- Example: api.internal.local

4 LOAD BALANCING

- A load balancer distributes incoming traffic across multiple backend servers (targets).
- It improves availability, reliability, and scalability.
- It performs health checks and only sends traffic to healthy servers.

5 LAYER 4 vs LAYER 7

FEATURE	LAYER 4 (L4)	LAYER 7 (L7)
Works at	Transport layer (TCP/UDP)	Application layer (HTTP/HTTPS)
Routes based on	IP address and port	URL, hostname, headers, etc.
Use cases	Simple, high-performance load balancing	Advanced routing, multiple applications
Examples	AWS NLB, Azure LB, Google TCP/UDP LB	AWS ALB, Azure Application Gateway, Google HTTP(S) LB

6 KEY LOAD BALANCER FEATURES

HEALTH CHECKS
Continuously check if backend targets are healthy.

BACKEND TARGETS
Servers, containers, or services that receive traffic.

TLS TERMINATION
Handles SSL/TLS encryption so your applications don't have to.

APPLICATION GATEWAYS
Provide advanced routing, SSL offloading, WAF, and application-level features.

7 REQUEST FLOW

8 TROUBLESHOOTING UNHEALTHY TARGETS

1. Check target health status in the load balancer console.
2. Verify the application is running on the correct port.
3. Check security group / firewall rules.
4. Look at application logs for errors.
5. Confirm the health check path (e.g, /health) returns the expected response.
6. Test connectivity from the load balancer to the target.

9 JUNIOR ENGINEER TIP

Understand how DNS, load balancers, and networking work before memorizing cloud provider service names. The concepts are the same across AWS, Azure, Google Cloud and other platforms.

Game Skills More Opportunities

CLOUD STORAGE
OBJECT, BLOCK AND FILE

STORE. PROTECT. SCALE. ANYWHERE.

Same Knowledge Bigger Opportunities

1 WHY CLOUD HAS DIFFERENT STORAGE TYPES

- Applications store different types of data and have different performance, access and scalability requirements.
- No single storage type is best for everything.
- Cloud providers offer multiple storage options to balance cost, performance, durability and use cases.

Choose the right storage type based on your application needs, not just the price.

2 OBJECT STORAGE

- Stores data as objects (includes data, metadata and a unique ID).
- Accessed over HTTP/HTTPS.
- Highly scalable and durable.
- Best for unstructured data (images, videos, backups, logs).
- Examples: AWS S3, Azure Blob Storage, Google Cloud Storage.

3 BLOCK STORAGE

- Provides raw block-level storage (like a virtual hard drive).
- Low latency and high performance.
- Used for virtual machines, databases and applications that need high IOPS.
- Examples: AWS EBS, Azure Managed Disks, Google Persistent Disk.

4 FILE STORAGE

- Provides shared file storage accessible by multiple servers.
- Supports standard file protocols (NFS, SMB).
- Used for applications that need a shared file system (e.g. content management, home directories).
- Examples: AWS EFS, Azure Files, Google Filestore.

5 PERSISTENT vs EPHEMERAL STORAGE

PERSISTENT STORAGE

- Data remains even if the instance is stopped or terminated.
- Used for databases, important application data and long-term storage.
- Examples: EBS, EFS, Persistent Disk, Filestore, S3.

EPHEMERAL STORAGE

- Data is temporary and lost when the instance is stopped or terminated.
- Used for caches, temporary files, scratch data.
- Examples: Instance store, temporary disks in VMs, /tmp directories.

6 CLOUD PROVIDER STORAGE SERVICES

USE CASE	AWS	AZURE	GOOGLE CLOUD
Object Storage	S3	Blob Storage	Cloud Storage
Block Storage	EBS	Managed Disks	Persistent Disk
File Storage	EFS	Azure Files	Filestore

7 DURABILITY vs AVAILABILITY

DURABILITY

- Protection against data loss over time.
- Measures how likely your data is to be permanently lost.
- Example: AWS S3 offers 11 nines (99.999999999%) durability.

AVAILABILITY

- How quickly your data is accessible when needed.
- Measures the uptime and accessibility of the storage service.
- Example: S3 Standard offers 99.99% availability.

8 STORAGE CLASSES

- Different pricing tiers for different use cases.
- Examples (AWS S3): Standard, Intelligent-Tiering, Glacier, Deep Archive.
- Examples (Azure): Hot, Cool, Archive.
- Examples (Google Cloud): Standard, Nearline, Coldline, Archive.
- Use lower-cost classes for infrequent access data.

9 LIFECYCLE POLICIES

- Automatically move data between storage classes based on age or access patterns.
- Helps reduce costs.
- Example: Move logs to cheaper storage after 30 days.
- Can also automatically delete data after a defined period.

10 ENCRYPTION

- Protects your data at rest and in transit.
- Most cloud storage services offer encryption by default.
- You can use provider-managed keys or your own keys (KMS).
- Always enable encryption for sensitive data.

11 CHOOSING THE CORRECT STORAGE TYPE

- What type of data are you storing? (structured, unstructured, or file-based)
- Do you need high performance or low cost?
- How long do you need to keep the data?
- Do multiple servers need to access the data?
- What are your durability and availability requirements?
- Will the data grow over time?
- Are there compliance or security requirements?
- Use a combination of storage types in real-world architectures.

"Right storage.
Right cost.
Right performance.
Right architecture."

CLOUD DATABASES AND MANAGED DATA SERVICES

Better Engineers Brighter Futures

RELIABLE DATA. SCALABLE APPLICATIONS. REAL-WORLD IMPACT.

1 DATABASE ON A VM vs MANAGED DATABASE

DATABASE ON A VM	MANAGED DATABASE
• You install and manage the database software. • You handle patching, backups, scaling and high availability. • More control but more operational work. • Use when you have specific custom requirements.	• The cloud provider manages installation, patching, backups, scaling and high availability. • You focus on using the database, not managing infrastructure. • More reliable and easier to scale. • Examples: AWS RDS, Azure SQL Database, Google Cloud SQL.

2 RELATIONAL DATABASES (SQL)

- Store data in structured tables (rows and columns).
- Support SQL (Structured Query Language).
- Good for transactional data and complex queries.
- Examples: MySQL, PostgreSQL, Oracle, Microsoft SQL Server, Amazon RDS, Azure SQL, Google Cloud SQL.

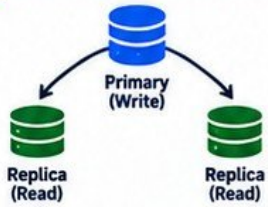
3 NoSQL DATABASES

- Store data in flexible formats (document, key-value, column, or graph).
- Designed for high scale and flexible schemas.
- Good for large volumes of unstructured or semi-structured data.
- Examples: MongoDB, DynamoDB, Cassandra, Redis, Azure Cosmos DB, Google Firestore.

4 MANAGED DATABASE SERVICES

AWS	Azure	Google Cloud
• Amazon RDS (MySQL, PostgreSQL, SQL Server, Oracle) • Amazon Aurora • Amazon DynamoDB • Amazon ElastiCache	• Azure SQL Database • Azure Database for PostgreSQL • Azure Database for MySQL • Azure Cosmos DB • Azure Cache for Redis	• Cloud SQL (MySQL, PostgreSQL, SQL Server) • Cloud Spanner • Firestore • Bigtable • Memorystore (Redis)

5 PRIMARY AND REPLICAS



- Primary handles write operations (inserts, updates, deletes).
- Replicas handle read operations.
- Improves performance and availability.

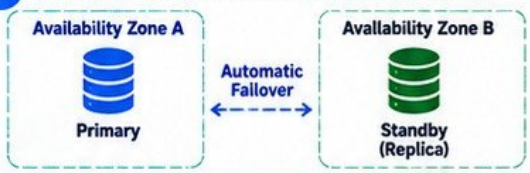
6 BACKUPS

- Automatically create backups of your database.
- Helps you recover from accidental deletions, corruption or failures.
- Most cloud providers offer automated and on-demand backups.
- Always test your backups.

7 READ REPLICAS

- Read replicas are copies of the primary database.
- Used to offload read traffic.
- Improve performance and scalability.
- Can be in the same region or a different region.

8 MULTI-ZONE DEPLOYMENT AND FAILOVER



- Databases can be deployed across multiple availability zones for high availability.
- If the primary fails, traffic is automatically routed to a standby replica.
- Reduces downtime and increases resilience.

9 CONNECTION ENDPOINTS

Example Endpoints:
AWS RDS: mydb.abc123.rds.amazonaws.com
Azure SQL: mydb.database.windows.net
Google Cloud SQL: mydb.gcp.example.com

- You connect to the database using an endpoint (DNS name).
- The endpoint routes you to the correct database instance.
- Different endpoints may exist for primary, read replicas, or cluster endpoints.

10 DATABASE NETWORKING

- Databases should be in private subnets.
- Control access using security groups or firewall rules.
- Allow only required ports (e.g. 3306, 5432).
- Use VPC peering or private links for secure access.
- Do not expose databases to the internet.

11 CONNECTION POOLING

- Connection pooling reuses existing database connections.
- Improves application performance and reduces load on the database.
- Commonly used in web applications and microservices.

12 PRODUCTION PROBLEM: APPLICATION CANNOT CONNECT TO ITS DATABASE

Common Causes:

- Wrong endpoint or port
- Security group / firewall blocking traffic
- Database not running
- Network connectivity issue (VPC, subnet, route table)
- Incorrect credentials
- Database is in a private subnet and not accessible
- Connection limit reached
- DNS resolution issue

Troubleshooting Steps:

1. Check the database endpoint and port.
2. Verify security group / firewall rules.
3. Confirm the database is running.
4. Test network connectivity (e.g. ping, telnet).
5. Verify credentials and permissions.
6. Check if the database is in a private subnet.
7. Look at database logs.
8. Check connection limits and use connection pooling.

KEY TAKEAWAY

A well-designed database architecture is reliable, secure, scalable, and backed up. Always understand the network, access, and failover design before going to production.

IAM, IDENTITIES, ROLES AND LEAST PRIVILEGE

Same Knowledge Bigger Opportunities

SECURE ACCESS. RIGHT PEOPLE. RIGHT PERMISSIONS. RIGHT PURPOSE.

1 IDENTITY AND ACCESS MANAGEMENT (IAM)

- IAM is a service that helps you control who can access your cloud resources and what they can do.
- You define identities (users, groups, roles) and permissions using policies.
- It is a core part of cloud security.

IAM ensures the right people and services have the right access to the right resources at the right time.

2 AUTHENTICATION vs AUTHORIZATION

AUTHENTICATION	AUTHORIZATION
Proves who you are.	Determines what you are allowed to do.
Uses passwords, MFA, access keys, or tokens.	Uses policies and permissions.
Answers: "Are you who you say you are?"	Answers: "What can you do?"

First you authenticate, then you are authorized to perform specific actions.

3 USERS

- A user is an identity with its own credentials (password, access key, etc).
- Used for humans (e.g. DevOps engineer, admin, auditor).
- Should be assigned to groups rather than individual permissions.

4 GROUPS

- A group is a collection of users.
- Permissions are attached to the group.
- Users in the group inherit the permissions.
- Makes access management easier at scale.

5 ROLES

- A role is an identity with permissions that can be assumed (temporary access).
- Used for humans or services.
- Ideal for cross-account access and workload access.
- No long-term credentials required.

6 SERVICE IDENTITIES

- Non-human identities used by applications, services or workloads.
- Examples: EC2 instance roles, Kubernetes service accounts, Lambda roles, Azure managed identities, Google service accounts.
- Used for secure, automated access to cloud resources.

7 POLICIES

- A policy is a set of rules that define permissions.
- Written in JSON.
- Specifies what actions are allowed or denied on which resources.
- Can be attached to users, groups, or roles.

8 PERMISSIONS

- Permissions define the actions an identity can perform.
- Examples: read, write, delete, create, list.
- Use least privilege (only the permissions needed).

9 ALLOW AND DENY

ALLOW	DENY
Grants permission to perform an action on a resource.	Explicitly blocks an action.
Defined in policies with "Allow".	Deny always overrides allow.
	Useful for security restrictions.

10 TEMPORARY CREDENTIALS

- Short-lived credentials used for secure access.
- Expire automatically after a set time.
- Reduce the risk of credential leakage.
- Used with roles, federated login, and workload identity.

11 WORKLOAD IDENTITY

- Allows applications and workloads to securely access cloud resources without hardcoded keys.
- Examples: EC2 instance roles, EKS IRSA, Azure managed identity, Google Workload Identity.
- More secure than static access keys.

12 LEAST PRIVILEGE

- Give only the minimum permissions required to perform a task.
- Reduces the risk of accidental or malicious damage.
- Regularly review and remove unnecessary permissions.
- Apply least privilege to both human and workload identities.

13 MFA (MULTI-FACTOR AUTHENTICATION)

- Adds an extra layer of security.
- Usually a password plus a code from an app (e.g. authenticator app).
- Strongly recommended for all human users.

14 WHY ACCESS KEYS SHOULD NOT BE HARDCODED

- Hardcoded keys can be exposed in code repositories, logs or configuration files.
- Can be accidentally shared or leaked.
- Use roles, environment variables, or secret management tools instead.

15 SECURITY REMINDER

Human identity and workload identity are different problems.

Humans use MFA and personal or federated accounts.
Workloads use roles and service identities.
Secure both.

Learn Build Share VERIQTA

SECURE TODAY. HIGHER TOMORROWS.

Better Engineers Brighter Futures

SCALING, ELASTICITY AND HIGH AVAILABILITY

Build systems that grow, stay available and handle real-world traffic.

1 VERTICAL SCALING

- Increase the size of a single server (CPU, memory, storage).
- Good for simple workloads.
- Limited by the maximum size of the instance.
- Does not improve high availability on its own.

Example: Move from 2 vCPU, 4 GB RAM to 8 vCPU, 16 GB RAM.

2 HORIZONTAL SCALING

- Add more servers (instances) to share the load.
- Handles more traffic.
- Improves high availability.
- Common in cloud environments.
- Works best for stateless applications.

Example: Scale from 2 instances to 6 instances.

3 AUTO SCALING

- Automatically adds or removes instances based on demand.
- Uses metrics like CPU, memory or custom metrics.
- Helps you handle traffic spikes.
- Reduces cost when demand is low.
- Keeps applications available.

Example: Add instances when CPU > 70%, remove when CPU < 30%.

4 SCALING CONFIGURATION

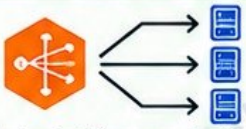
Minimum capacity	The smallest number of instances.
Desired capacity	The target number of instances.
Maximum capacity	The largest number of instances.

5 SCALING METRICS

CPU Usage | Memory Usage | Request Count | Custom Metrics

Example: Scale out when CPU > 70% for 5 minutes.

6 LOAD BALANCING



- Distributes traffic across instances.
- Improves availability and performance.
- Works with auto scaling.
- Health checks remove unhealthy instances.

7 STATELESS APPLICATIONS

- Each request is independent.
- Instances can be added or removed easily.
- Ideal for horizontal scaling.
- Examples: web apps, APIs, microservices.

8 SESSIOG CONSIDERATIONS

- Avoid storing user sessions on a single instance.
- Use shared stores (Redis, database) or stateless tokens (JWT).
- Ensures users stay logged in when instances change.

9 MULTI-ZONE ARCHITECTURES

- Deploy instances across multiple Availability Zones (AZs).
- Protects against zone failures.
- Improves availability and resilience.
- Use a load balancer across AZs.

10 REMOVE SINGLE POINTS OF FAILURE

- Avoid relying on one server, one AZ, or one database.
- Use multiple instances and AZs.
- Use managed services where possible.
- Design for failure from the beginning.

11 EXAMPLE ARCHITECTURE

Load Balancer → Auto Scaling → Instances across multiple AZs

JUNIOR ENGINEER TIP

Scaling and high availability are not just about adding more servers. Design for stateless applications, use multiple zones, and automate with auto scaling to handle real-world traffic.

LEARN BUILD GROW WITH VERIQTA

CONTAINERS AND KUBERNETES IN THE CLOUD

Package. Deploy. Scale. Run Applications Anywhere.

1 VM vs CONTAINER

VIRTUAL MACHINE (VM)	CONTAINER
App	App
OS	Container Runtime
Virtual Hardware	Host OS
Physical Server	Physical Server
• Runs a full operating system • Heavier and larger • Slower to start • Good for diverse workloads	• Shares the host OS • Lightweight and smaller • Starts in seconds • Ideal for modern applications (microservices)

2 CONTAINER IMAGES

- A container image is a read-only package that contains your application, dependencies and configuration.
- Built from a Dockerfile.
- Portable and consistent across environments.
- Use version tags (for example: v1.0.0) instead of latest in production.

3 CONTAINER REGISTRIES

- Stores container images so they can be downloaded and deployed.
- Public registries: Docker Hub, Amazon ECR Public, etc.
- Private registries: AWS ECR, Azure Container Registry (ACR), Google Artifact Registry.
- Use image scanning to check for vulnerabilities.

4 MANAGED CONTAINER SERVICES

Amazon ECS	Run containers without managing servers.
Azure Container Apps	Serverless containers for modern applications.
Google Cloud Run	Serverless containers with automatic scaling.

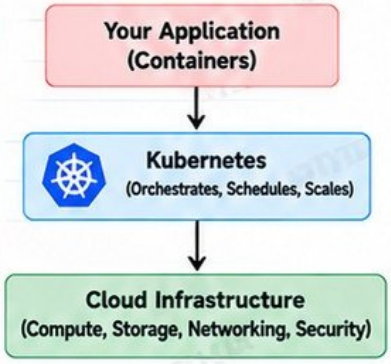
5 MANAGED KUBERNETES

Amazon EKS	Managed Kubernetes on AWS.
Azure AKS	Managed Kubernetes on Azure.
Google GKE	Managed Kubernetes on Google Cloud.

6 KEY KUBERNETES CONCEPTS

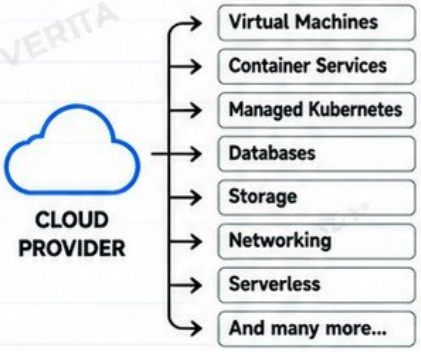
Node	A worker machine that runs your containers.
Pod	Runs one or more containers.
Service	Provides stable network access to pods.
Load Balancer	Distributes traffic to applications across pods.

7 WHERE KUBERNETES FITS



- Kubernetes runs on top of cloud infrastructure (virtual machines or managed services).
- It manages your container workloads across multiple nodes.
- You still use cloud services for networking, storage, load balancers, security, and more.

8 CLOUD DOES NOT EQUAL KUBERNETES



- The cloud is a broad platform that offers many services.
- Kubernetes is just one part of the cloud.
- You can run applications on VMs, serverless, or managed container services without using Kubernetes.
- Choose the right tool for your application and requirements.

9 JUNIOR ENGINEER TIP

Learn the core concepts of containers and Kubernetes, but also understand the cloud services around them. Production applications use many services together: compute, networking, storage, security, monitoring and more.

LEARN
PRACTICE
BUILD
GROW
WITH VERIQTA

INFRASTRUCTURE AS CODE

STOP BUILDING EVERYTHING BY HAND

Define. Automate. Version. Deploy. Repeat.

1 WHAT IS INFRASTRUCTURE AS CODE (IaC)?

- Infrastructure as Code (IaC) means you define and manage infrastructure using code instead of manual steps.
- You write configuration files that describe the resources you want.
- The cloud provider then creates the infrastructure for you.
- It makes infrastructure repeatable, consistent and version controlled.

2 MANUAL PROVISIONING PROBLEMS

- Time consuming
- Inconsistent environments
- Prone to human error
- Hard to reproduce
- Difficult to track changes
- No easy rollback
- Not scalable
- Leads to configuration drift

3 DECLARATIVE INFRASTRUCTURE

- You describe the desired state of your infrastructure.
- You do not write step-by-step commands.
- The tool figures out what needs to be created, updated or deleted.
- You focus on what you want, not how to do it.

4 POPULAR IaC TOOLS

Terraform

- Multi-cloud
- Open source
- Widely used

OpenTofu

- Community driven
- Open source
- Terraform compatible

AWS CloudFormation

- Native to AWS
- Uses JSON or YAML
- Good for AWS-only

Azure Bicep

- Native to Azure
- Uses Bicep language
- Simple and modern

5 KEY CONCEPTS

Plan	Shows what will be created, changed or destroyed.
Apply	Creates or updates the infrastructure.
State	Keeps track of the current infrastructure.
Drift	When real infrastructure does not match the code.
Modules	Reusable code for common infrastructure.
Version Control	Store your IaC code in Git.
Review	Always review changes before applying.

6 VERSION-CONTROLLED INFRASTRUCTURE

- Store your IaC code in GitHub, GitLab or similar.
- Track changes over time.
- Use branches for new changes.
- Review with pull requests.
- Keeps your infrastructure auditable.
- Makes collaboration easy.
- Allows rollback to previous versions.

7 IaC WORKFLOW

Git → Code is committed to a repository.

→ **IaC** → Define infrastructure in code (Terraform, OpenTofu, CloudFormation, or Bicep).

→ **Review** → Run plan, review changes, get approval (pull request).

→ **Cloud API** → Apply the configuration and the tool calls the cloud provider API.

→ **Infrastructure** → Resources are created or updated in the cloud.

8 EXAMPLE: TERRAFORM WORKFLOW

```
terraform init    # Initialize the working directory
terraform plan    # Preview changes
terraform apply   # Create or update infrastructure
terraform destroy # Remove resources (use with caution)
```

9 JUNIOR ENGINEER TIP

Start with a small project.
Use version control.
Always run plan before apply.
Use modules to keep your code clean.
Treat your infrastructure code like application code.

10 REMEMBER

Automate.
Review changes.
Keep it in Git.
Avoid drift.
Use reusable modules.
Build, test and deploy with confidence.

INFRASTRUCTURE AS CODE TURNS INFRASTRUCTURE INTO A REPEATABLE, SCALABLE AND RELIABLE PROCESS.

LEARN
BUILD
GROW
WITH VERIQTA

MONITORING, LOGGING AND OBSERVABILITY

See What's Happening. Find the Cause. Keep Systems Healthy.

1 WHAT IS OBSERVABILITY?

Observability is the ability to understand what is happening inside your system using metrics, logs, traces and events. It helps you detect, troubleshoot and fix problems before users are affected.

"You can't manage what you can't see."

2 THE FOUR PILLARS

Metrics	Numeric data over time (CPU, memory, requests).
Logs	Detailed event records (application and system).
Traces	Follow a request across multiple services.
Events	Important changes in the system (restarts, deployments, scaling).

3 CLOUD MONITORING

Cloud providers offer managed monitoring services to collect and visualize metrics, logs and other telemetry data.

AWS CloudWatch
Metrics, logs, alarms, dashboards and more.

Azure Monitor
Metrics, logs, traces, alerts and application insights.

Google Cloud Observability
Metrics, logs, traces, error reporting and dashboards.

4 KEY METRICS TO MONITOR

CPU	CPU utilization, throttling, load average.
Memory	Memory usage, available memory, memory pressure.
Disk	Disk usage, IOPS, read/write latency.
Network	Network in/out, errors, latency.
Application Metrics	Request count, response time, error rate, active users.

5 DASHBOARDS AND ALERTS

Dashboards

- Visualize metrics, logs and traces.
- Track system health in real time.
- Create separate dashboards for infrastructure, applications and business metrics.

Alerts

- Get notified when something is not right.
- Use thresholds (for example CPU > 80 percent).
- Send alerts to email, Slack, PagerDuty, etc.
- Reduce alert noise (avoid too many false alerts).

6 LOGS, TRACES AND EVENTS

Logs
- Application logs
- System logs
- Container logs
- Use log aggregation and search.

Traces
- Show the full request path across services.
- Identify where latency occurs.
- Useful in microservices and distributed systems.

Events
- Resource changes
- Deployments
- Scaling activities
- System failures
- Useful for audit and troubleshooting.

7 SYMPTOMS vs CAUSES

SYMPTOMS
- Application is slow
- Errors in the logs
- High CPU usage
- Pods keep restarting
- Users cannot access the service
- Increased response time

CAUSES
- High traffic
- Resource limits too low
- Memory leaks
- Database slow
- Network issues
- Misconfiguration
- Recent deployment change

8 TROUBLESHOOTING MODEL

Observe → **Investigate** → **Hypothesize** → **Verify**

Observe
Look at metrics, logs, traces and events.

Investigate
Find what changed and gather more data.

Hypothesize
Identify possible causes.

Verify
Test your theory, confirm the root cause and fix it.

9 EXAMPLE DASHBOARD

10 JUNIOR ENGINEER TIP

- Start with key metrics (CPU, memory, errors, response time).
- Learn to read logs.
- Use dashboards and alerts early.
- Always look for the cause, not just the symptom.
- Practice troubleshooting in real scenarios.

11 REMEMBER

- Monitoring helps you know something is wrong.
- Observability helps you understand why.
- Good systems are measured, not guessed.

OBSERVE. INVESTIGATE. HYPOTHESIZE. VERIFY. THAT'S HOW YOU KEEP PRODUCTION RUNNING.

CLOUD SECURITY AND THE SHARED RESPONSIBILITY MODEL

Security in the cloud is a shared responsibility. The cloud provider secures the cloud, but you are responsible for what you build, configure, and run in it.

1 PROVIDER RESPONSIBILITY

The cloud provider is responsible for securing the underlying infrastructure that runs the cloud.

- Physical data centers
- Hardware
- Networking infrastructure
- Virtualization layer
- Availability and resilience
- Physical security

CLOUD PROVIDER

2 CUSTOMER RESPONSIBILITY

You are responsible for securing what you create, configure, and run in the cloud.

- Operating systems
- Applications
- Data and encryption
- Identity and access management
- Network configuration
- Security groups and firewalls
- Secrets management
- Monitoring and logging
- Compliance and governance

YOU (CUSTOMER)

3 SHARED RESPONSIBILITY MODEL

SECURITY OF THE CLOUD

Provider secures the infrastructure.

SECURITY IN THE CLOUD

You secure your workloads, configurations, data and access.

4 KEY CLOUD SECURITY AREAS

Identity Security
Users, roles, permissions, MFA

Network Security
VPC, subnets, firewalls, traffic control

Encryption at Rest
Protect data stored in disks, databases, storage

Encryption in Transit
Use TLS to protect data in transit

Key Management
Create, rotate and control encryption keys

Secrets Management
Store and access secrets securely (e.g. AWS Secrets Manager)

Vulnerability Management
Scan and patch OS, containers and dependencies

Security Logging
Monitor, audit and detect suspicious activity

5 COMMON CLOUD SECURITY RISKS

PUBLIC EXPOSURE

- Unintentionally public storage
- Open security group rules (0.0.0.0/0)
- Exposed databases or management ports!

MISCONFIGURATION

- Wrong IAM permissions
- Unrestricted network access
- Default settings left unchanged
- Exposed secrets
- Insecure container images

6 WHY "THE CLOUD PROVIDER SECURES EVERYTHING" IS DANGEROUS THINKING

- The provider does NOT secure your applications.
- The provider does NOT manage your data and access.
- The provider does NOT prevent misconfigurations.
- The provider does NOT know your security requirements.
- You can still be compromised due to your own mistakes.
- You are accountable for your environment and data.

7 JUNIOR ENGINEER TIP

Use the shared responsibility model as a checklist.
Always ask: What is the provider's job and what is my job?
Security is a daily habit, not a one-time setup.

8 REMEMBER

The cloud gives you powerful tools.
With great power comes great responsibility. Secure what you build.

Cloud Skills Better Opportunities

CLOUD COST, FINOPS AND RESOURCE MANAGEMENT

Learn Build Grow

Run reliable, scalable and cost-efficient infrastructure.

1 PAY-AS-YOU-GO

- Pay only for what you use
- No large upfront cost
- Scale up or down anytime
- Ideal for dynamic workloads
- You are billed based on actual usage (compute, storage, network, etc.)

2 MAJOR COST CATEGORIES

Compute Pricing

- VM instances
- vCPU and RAM
- Instance types
- On-demand, reserved, spot

Storage Pricing

- Object storage
- Block storage
- File storage
- Storage classes
- Backup storage

Network Egress

- Data leaving the cloud
- Can be expensive
- Varies by region
- Keep data close when possible

3 COMMON COST CHALLENGES

Idle Resources

- Running but not in use
- Wastes money
- Stop or remove when not needed

Overprovisioning

- Larger instances than required
- Paying for unused capacity
- Rightsize based on real usage

Unmanaged Sprawl

- Untracked resources
- Old environments
- Forgotten storage
- Resources no longer needed

4 COST OPTIMIZATION OPTIONS

Reserved Capacity

- Commit for 1–3 years
- Lower pricing
- Good for steady workloads

Savings Commitments

- Flexible across services
- Lower cost
- Good for changing workloads

Spot / Preemptible Compute

- Large cost savings
- For non-critical workloads
- Instances can be terminated anytime

5 TAGS AND LABELS

- Add tags to all resources
- Example: Environment, Project, Owner
- Helps with cost tracking and reporting
- Required for good FinOps and governance

6 BUDGETS AND COST ALERTS

- Set monthly or quarterly budgets
- Get alerts when you reach a threshold
- Prevent unexpected bills
- Available in AWS, Azure and Google Cloud

7 RIGHTSIZING

- Use monitoring data (CPU, memory, etc.)
- Choose the right instance type and size
- Remove unused resources
- Continuously review and adjust

8 FINOPS FUNDAMENTALS

- Visibility: Know what you are spending on
- Accountability: Tag and assign ownership
- Optimization: Remove waste and rightsize
- Continuous improvement: Review regularly
- Collaboration: Engineers, finance and business teams

9 PRODUCTION AWARENESS

A technically working architecture can still be financially broken.

Your application may run perfectly, but if it is overprovisioned, not right-sized, or poorly managed, it can waste a lot of money. Good engineering includes cost awareness.

Build Efficiently Scale Sustainably

Learn
Plan
Practice
Recover

BACKUP, DISASTER RECOVERY AND RESILIENCE

Resilient
Systems
Stronger
Businesses

Protect your data. Prepare for failures. Keep your business running.

1 BACKUP vs HIGH AVAILABILITY

BACKUP

- A copy of your data at a point in time
- Used to recover from data loss (accidental delete, corruption, etc.)
- Not the same as high availability
- Usually stored separately

HIGH AVAILABILITY

- Keeps your application running during failures
- Uses multiple servers (zones or regions)
- Reduces downtime
- Does not replace backups
- Protects against infrastructure failures

2 DISASTER RECOVERY

A disaster recovery (DR) strategy helps you recover your entire application and data after a major outage, such as:

- Regional failure
- Natural disaster
- Large scale service outage
- Accidental deletion
- Security incident (ransomware)

3 RPO AND RTO

RPO (Recovery Point Objective)

- How much data you can afford to lose.
- Example: RPO = 1 hour means you may lose up to 1 hour of data.

RTO (Recovery Time Objective)

- How quickly you need to recover.
- Example: RTO = 4 hours means the service must be restored within 4 hours.

4 KEY TECHNOLOGIES

Snapshots
Point-in-time copies of disks or volumes

Database Backups
Automated backups for managed databases

Replication
Copies data to another location in real time or near real time

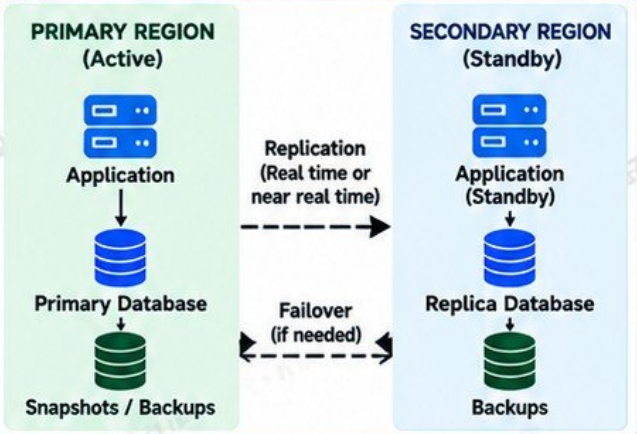
Multi-Zone
Resources spread across multiple availability zones

Multi-Region
Resources replicated to different regions for disaster recovery

Active-Passive
One active, one standby (takes over on failure)

Active-Active
Both regions active and serving traffic

5 ARCHITECTURE EXAMPLE



6 RESTORE TESTING

- Regularly test your backups.
- Verify that you can restore data and applications.
- Check recovery time and process.
- A backup that has never been restored is just an assumption.

7 FAILURE DOMAINS

- Zones can fail (e.g. power, network).
- Regions can fail (e.g. natural disaster).
- Design for multiple failure domains.
- Avoid single points of failure.

8 DISASTER RECOVERY RUNBOOKS

- Document step-by-step recovery procedures.
- Include who is responsible.
- Test the runbook regularly.
- Keep it up to date.
- Ensure all team members understand it.

9 JUNIOR ENGINEER TIP

A backup you have never restored is an assumption.
Always test your recovery process.

VERIQTA

PAGE 19

TROUBLESHOOTING CLOUD INFRASTRUCTURE

Learn
Troubleshoot
Solve
Grow

Same
Skills
Bigger
Opportunities

REAL INCIDENTS. REAL STEPS. REAL SOLUTIONS.

1 VM CANNOT REACH THE INTERNET

Check these components in order:

- **Routes** — Check route table entries
- → **Gateway** — Internet gateway or NAT gateway
- → **Firewall** — Security groups or network ACLs
- → **IP Configuration** — Public/private IP and subnet

COMMON CAUSES

- No route to internet
- Missing or misconfigured gateway
- Firewall blocking outbound traffic
- Instance in private subnet without NAT
- Incorrect IP configuration

2 USER CANNOT REACH APPLICATION

Check these components in order:

- **DNS** — Domain resolves to correct IP
- → **Load Balancer** — Healthy and correct listeners
- → **Firewall** — Allow traffic on required ports
- → **Backend** — Targets are healthy
- → **Application** — Application is running correctly

COMMON CAUSES

- DNS pointing to wrong endpoint
- Load balancer with no healthy targets
- Firewall blocking traffic
- Application down or misconfigured
- Incorrect security group rules

3 APPLICATION CANNOT REACH DATABASE

Check these components in order:

- **DNS** — Database endpoint resolves correctly
- → **Port** — Correct port is open
- → **Network Path** — Route and subnet configuration
- → **Firewall** — Allow traffic from application
- → **Credentials** — Valid username and permissions
- → **Database State** — Database is running and accepting connections

COMMON CAUSES

- DNS resolution issues
- Wrong or closed port
- Network routing problems
- Firewall blocking traffic
- Invalid credentials
- Database not running or overloaded

4 APPLICATION IS SUDDENLY SLOW

Check these components in order:

- **Metrics** — Check CPU, memory, latency
- → **Compute** — Instance/cluster performance
- → **Database** — Slow queries or high load
- → **Storage** — High I/O or latency
- → **Network** — Network latency or packet loss
- → **Application** — Application logs and performance

COMMON CAUSES

- High CPU or memory usage
- Database performance issues
- Storage latency or limits
- Network latency or packet loss
- Application errors or inefficient code

5 DEPLOYMENT WORKS IN STAGING BUT FAILS IN PRODUCTION

Check these components in order:

- **Configuration** — Compare env configuration
- → **IAM** — Permissions and roles
- → **Network** — VPC, subnets, firewalls
- → **Secrets** — Environment variables, secrets
- → **Dependencies** — External services and versions
- → **Capacity** — Instance size, limits, quotas

COMMON CAUSES

- Different configuration values
- Missing IAM permissions
- Network restrictions
- Missing or incorrect secrets
- Different dependency versions
- Insufficient capacity or quotas

TROUBLESHOOTING MINDSET

- Read logs before guessing
- Identify what changed
- Determine the failure boundary
- Test one hypothesis at a time
- Fix the cause
- Verify recovery

> Most production problems are not solved by luck, but by a structured troubleshooting process.
>
> VERIQTA

JUNIOR ENGINEER TIP

A working architecture does not always mean a healthy architecture. Always monitor, validate and test end-to-end.

FINAL PRODUCTION LAB

BUILD A CLOUD APPLICATION FROM ZERO

Build
Deploy
Learn
Troubleshoot
Grow

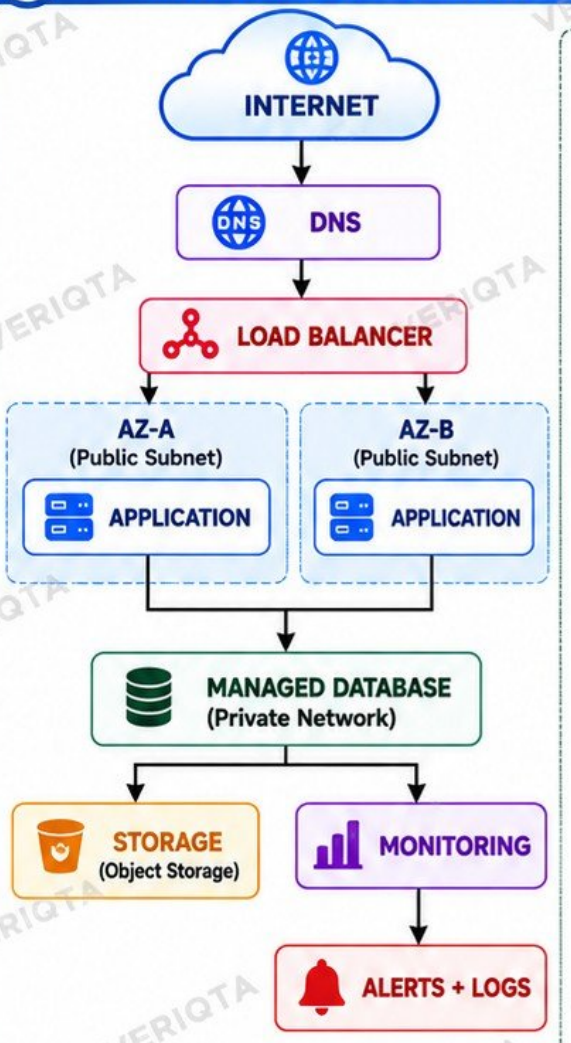
Same
Skills
Bigger
Opportunities

Bring everything together. Design. Deploy. Secure. Monitor. Scale. Recover.

1 LAB STEPS

1	Create a virtual network
2	Define CIDR ranges
3	Create public and private subnets
4	Configure routing
5	Configure internet access
6	Create security rules
7	Deploy compute
8	Deploy application
9	Configure object storage
10	Create managed database
11	Configure IAM
12	Protect credentials
13	Configure DNS
14	Add a load balancer
15	Configure health checks
16	Enable monitoring and logging
17	Add scaling
18	Create backup strategy
19	Test a failure
20	Diagnose it
21	Recover the application
22	Verify the system

2 FINAL ARCHITECTURE



- **IAM** (Users, Roles, Policies)
- **SECURITY** (Security Groups, Firewalls, NACLs)
- **NETWORKING** (VPC, Subnets, Routes, IGW, NAT)
- **BACKUP & DR** (Snapshots, Replication)
- **OBSERVABILITY** (Metrics, Logs, Alerts)
- **COST MANAGEMENT** (Tags, Budgets, RightSizing)

A SECURE, SCALABLE AND RESILIENT CLOUD APPLICATION

3 KEY TAKEAWAYS

- Understand how all cloud components work together
- Follow a structured deployment process
- Apply security best practices
- Plan for failure and recovery
- Monitor, optimize and control costs
- Think like a production engineer

4 LAB SUCCESS CRITERIA

- Application is accessible via DNS and load balancer
- Runs in multiple availability zones
- Uses managed database and object storage
- Proper security and IAM configured
- Monitoring and logging enabled
- Backup strategy in place
- You can recover from a simulated failure

5 JUNIOR ENGINEER TIP

"A working application is the beginning. A resilient, secure, well-monitored and cost-optimized application is the goal."